INFORME DESAFÍO 1 IMPLEMENTACIÓN DE TETRIS EN C++

INTEGRANTES:
ISABELLA CARVAJAL
CC.1094903251
JUAN SOLARTE
CC.1081052976

CURSO:
INFORMATICA II

UNIVERSIDAD DE ANTIOQUIA
SEMESTRE 2026-1
MEDELLÍN-ANTIOQUIA
20/03/2026

Tabla de contenido

- Contextualización
- Análisis del problema
- Diseño del sistema
- Arquitectura del programa
- Algoritmos
- Problemas encontrados durante el desarrollo

1. CONTEXTUALIZACION:

El presente desafío consiste en desarrollar una versión simplificada del juego Tetris utilizando el lenguaje de programación C++. El sistema funcionará por consola y permitirá al usuario interactuar con el juego mediante comandos para mover y rotar piezas. A diferencia del Tetris tradicional, esta implementación no utiliza gráficos complejos ni animaciones en tiempo real. En su lugar, el estado del tablero se mostrará mediante caracteres en consola. El documento del reto exige precisamente una interfaz por consola, sin animaciones en tiempo real, donde el sistema muestre el tablero y procese la acción elegida por el usuario en cada turno.

El objetivo principal del proyecto es aplicar varios conceptos estudiados durante el curso de programación, como operadores a nivel de bits, manejo de memoria dinámica, uso de punteros, arreglos y programación modular. Además, se deben respetar restricciones importantes como no usar STL, no usar string y realizar la implementación en C++ dentro de Qt. Estas condiciones hacen que la solución no solo deba funcionar, sino que también esté bien pensada desde el punto de vista de la eficiencia y de la estructura del programa.

La idea general de la solución es representar tanto el tablero como las piezas mediante bits. Esto permite aprovechar operaciones como desplazamientos y máscaras para mover piezas, detectar colisiones y revisar si una fila está completa.

2. ANÁLISIS DEL PROBLEMA

Para desarrollar correctamente el juego es necesario dividir el problema en varias partes: representación del tablero, representación de las piezas, movimiento, rotación, detección de colisiones, eliminación de filas y condición de finalización.

2.1. Representación del tablero: comparando con la idea que se tenía desde un principio, se analizó que era necesario que el tablero contara con propiedades dinámicas, ya que sus dimensiones (alto, ancho) se definen de acuerdo con las instrucciones del usuario, y el contenido irá cambiando conforme el juego avanza, además, este contenido puede ser liberado cuando el juego acabe, también es conveniente en la sección de eliminación de filas, que analizará más adelante. Así mismo, se haría uso de punteros para trabajar sobre esta matriz bidimensional cómodamente pensando en usar un apuntador de apuntadores. También se pensó en la idea de que

el usuario pueda escoger dimensiones que no estén acordes, ya que estas deben ser de un mínimo de 8 y el ancho ha de ser múltiplo de 8, para facilitar la representación de los 8 bits acordes a las piezas y al tablero.

2.2. Representación de las piezas: en esta sección se tomó una idea que se tenía al principio, que es la representación de las piezas en un arreglo bidimensional estático, aunque previamente se había pensado en el uso de enteros para la representación individual de cada pieza, esta idea fue desechada, ya que el entero ocupa 4 bytes en memoria, mientras que una variable tipo char ocupa 1 byte, haciendo esta conversión se logra ver una mayor eficiencia en el uso de memoria. Sin embargo, durante la ejecución del juego se trabajará solo con una pieza representada por un arreglo estático de una dimensión, ya que el arreglo bidimensional estático solo tendrá lugar en el ámbito local de otra función. Ahora, también manteniendo una idea acorde a la inicial, cada elemento de la primera dimensión del arreglo será acorde a las diferentes 5 piezas, mientras que la segunda dimensión de tamaño 4 hará referencia a las 4 filas de cada pieza. Se consideró conveniente el uso de un arreglo para la representación de las piezas, ya que la estructura base de ellas no cambiará durante la ejecución y se sabe previamente su tamaño.

2.3 Movimiento: Para el movimiento de las piezas se pensó en el uso de las operaciones de bits << y >>, esta idea se retoma, sin embargo, no es tan directa como se planteó en un principio. En pro de la estructura de la idea original, es necesario saber previamente si el desplazamiento, sea derecha o izquierda tiene colisiones con otras fichas del tablero o con los bordes de este. Así que, para determinar si la ficha puede o no desplazarse hacia la izquierda o la derecha nos basamos en verificar la colisión de antemano, para luego así cambiar la posición de los bits, donde sus columnas son en las cuales se hace la operación. Ya que, en sí, aquí la pieza no cambia su forma, sino la posición de los bits dentro del arreglo, en sus columnas particularmente.

2.4 Rotación: Para la rotación es también necesario verificar la posible colisión entre la pieza próxima a rotar y el tablero, entonces se decide crear un arreglo con las dimensiones de la pieza para que sea una pieza de prueba, a ella se le llenarán sus filas en cero, para posteriormente compararla con la pieza original y hacer un cambio de coordenadas referentes a la rotación. Este cambio de coordenadas para la operación de rotar se hace ya que teniendo una matriz cuadrada de NxN se puede saber como queda distribuida si se aplica una rotación de 90 grados, donde sus

nuevas filas serán sus columnas anteriores y sus columnas serán N-1-I donde I representa el número de la fila. Una vez se haya hecho la rotación provisional de la pieza, se verificará la colisión de ella con el tablero actual, si no hay, se llena la pieza original con los valores de la pieza rotada. Aquí, aunque se dijo que el método de copia no era eficiente se decidió sacrificar este factor, ya que la matriz realmente es de un tamaño pequeño, que es unidimensional de tamaño 4 y su contenido son char, su espacio en memoria son 4 bytes, y al ser llenada el ciclo solo será de 4 iteraciones. Si se hubiera considerado el trabajar directamente con la pieza original, esta podría hacer la rotación perfectamente, sin embargo, sería tedioso si esta colisionara ya que tendría que volver a su valor original.

2.5 Detección de colisiones: Para la detección de colisiones se piensa en trabajar con una referencia inherente a cada ficha y dependiente del ancho, que servirá como el cero en la parte superior izquierda de la ficha, de esta manera se podrá recorrer la ficha en su totalidad y verificar si hay bits encendidos que puedan colisionar. Estas verificaciones se hacen con operaciones a nivel de bits que solicitan una coordenada para desplazar la mascara y lograr hacer la verificación del bit en particular. Esto se hace para cumplir con las operaciones a nivel de bits en el core de todo el programa, ya que para las diferentes acciones que solicite el usuario se necesita corroborar de antemano la colisión.

2.6 Eliminación de filas: La eliminación de filas ocurre cuando una fila del tablero está completamente ocupada. Esto se verifica con la función filallena, que revisa si todo los bytes de la fila tienen el valor 0xFF (Valor hexadecimal que representa un byte con todos sus bits en 1 en binario) y con esto el programa lo usa para comprobar que una fila está completamente llena ya que cada byte del tablero representa 8 celdas y entonces si un Byte es igual a 0xFF significa que las 8 posiciones están ocupadas lo que indica que todos sus bits están encendidos, la función limpiarfilas recorre el tablero desde la parte inferior hacia arriba y entonces cuando detecta fila llena elimina la fila liberando su memoria, desplaza todas las filas superiores una posición hacia abajo y finalmente crea una nueva fila vacía en la parte superior del tablero y así este proceso se repite en caso de que existan varias filas completas consecutivas, la limpieza de filas se ejecuta despues de fijar una nueva pieza en el tablero, permitiendo liberar espacio y mantener la jugabilidad.

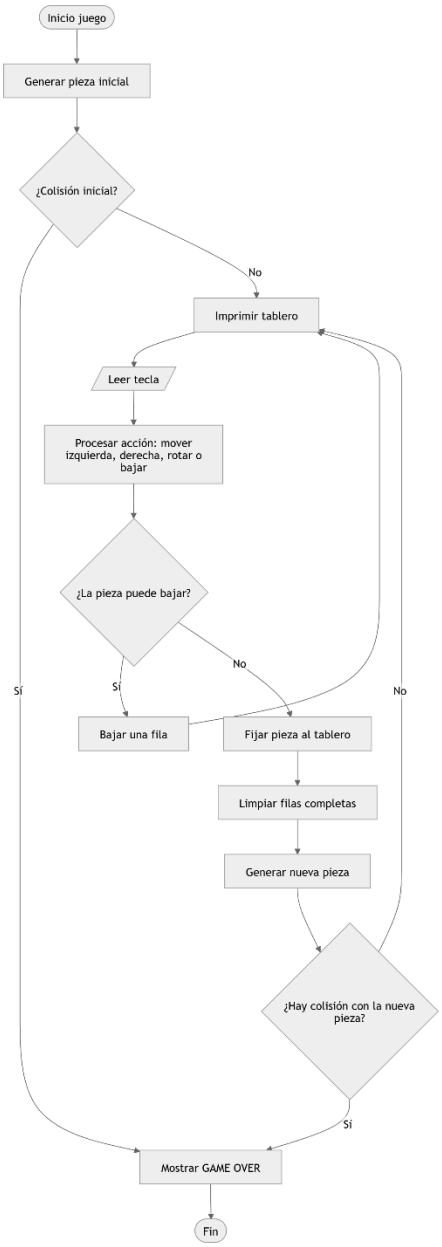
2.7 Condición de finalización: La condición de finalización del juego se controla mediante la variable gameOver, el juego termina cuando una nueva pieza no puede colocarse en su posición inicial sin generar una colisión, esto se verifica usando la función colisión la cual detecta si la pieza se sale de los límites del tablero, alcanza el fondo y verifica si se superpone con bloques ya existentes, después de fijar una pieza y limpiar filas se genera una nueva pieza en la parte superior del tablero y si esta nueva pieza colisiona inmediatamente, significa que no hay espacio disponible por lo que se activa gameOver y el juego finaliza.

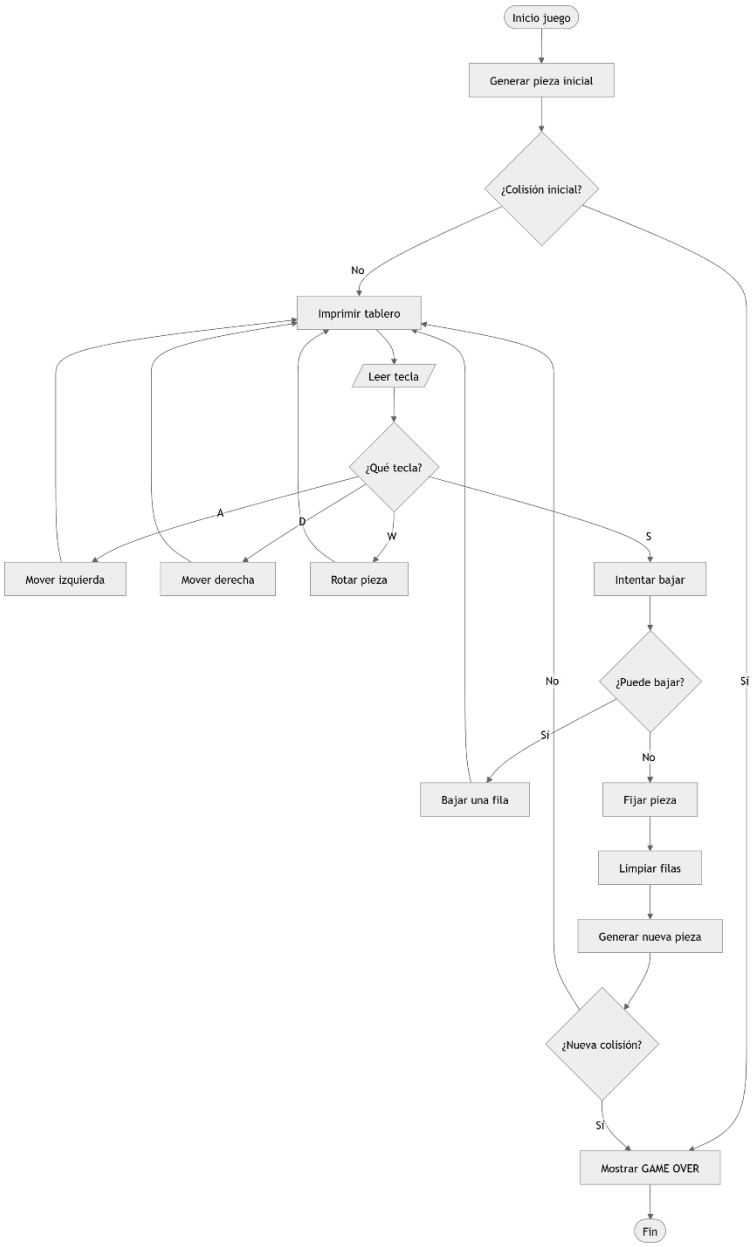
3. DISEÑO DEL SISTEMA

Diagramas de Flujo

Diagrama 1:

Diagrama 2:





4. *ARQUITECTURA DEL PROGRAMA*

4.1 Estructura general:

- Programa modular en archivos (main.cpp, Funciones.cpp, Funciones.h)
- Funciones atómicas: se trató de que cada función representara una acción específica para mejorar la legibilidad y el código contara con una estructura más procedimental.

4.2. Flujo del programa:

El flujo inicia en el programa principal **main,** donde se le pide al usuario que indique las dimensiones del tablero, posteriormente se analiza la validez de estas dimensiones. Si estas son certéras se procede a inicializar la variable tablero previamente declarada con la función **crearTablero** usando como argumentos el ancho y el alto ingresados por el usuario, de no ser así se retorna 1 para indicar que el programa no pudo ser ejecutado. Se hace después la invocación de la función **iniciarSemilla** que será de utilidad después para la generación de números aleatorios para la selección pseudoaleatoria de piezas. Luego, se invoca la función **juego** usando como parámetros el apuntador de apuntadores tablero, el ancho y el alto. Posteriormente, se realiza la liberación de memoria correspondiente a tablero con la función **liberarTablero.** Por último se retorna 0 para indicar que el programa fue ejecutado de forma exitosa.

5. ALGORITMOS

Nombre	Tipo	Parámetros	Descprición	Llama a
main	int	-	Realiza la primera interacción con el usuario para definir el alto y el ancho, además inicializa a la variable **tablero.	dimensionesValidas, iniciarSemilla,creartablero,juego ,liberarTablero
dimensionesValidas	bool	alto,ancho	Verifica que el alto sea de mínimo 8 y que el ancho sea de mínimo 8 y múltiplo de 8.	-
crearTablero	unsigned char **	alto, ancho	Reserva memoria para el tablero, llena su contenido en ceros.	-
juego	void	**tablero, ancho, alto	Se encarga de la lógica principal del programa con condicionales y ciclos.	generarPieza, colision, imprimirTablero, moverDer, moverIzq, intentarRot, bajar
liberarTablero	void	**Tablero, alto	Se encarga de liberar la memoria reservada para las dimensiones de tablero.	-
generarPieza	void	*pieza, &tipo	De acuerdo a un arreglo bidiemensional estático, genera un número aleatorio para la selección pseudoaleatora de la pieza, llenando esta con la pieza correspondiente al numero aleatorio y su contenido.	-
colision	bool	**tablero, alto, ancho, *pieza, fila, columna.	Recorre el contenido de la pieza en cuanto a sus filas y columnas, de acuerdo a verificaciones con otras	bitPieza,bitTablero

			funciones analiza superposiciones de fichas o colisiones en límites laterales e inferiores.	
bitPieza	bool	*pieza, fila, col	Realiza por medio de máscaras y operaciones a nivel de bits la verificación de un bit encendido en cierta posición del byte correspondiente a la pieza.	-
bitTablero	bool	**tablero, fila, columna	Realiza por medio de máscaras la verificación de bit encendido en cierta posición del byte para el tablero.	-
moverIzq	bool	**tablero, alto, ancho, *pieza, filapieza, &colpieza	Realiza cambios en colpieza -1 dependiendo si colisiona dada esta instrucción.	colision
moverDer	bool	**tablero, alto, ancho, *pieza, filapieza, &colpieza	Realiza cambios en colpieza +1 dependiendo si colisiona dada esta instrucción.	colision
imprimirTablero	void	**tablero, alto, ancho, *pieza, filapieza, colpieza	Recorre el tablero, hace verificaciones respecto a bits encendidos en pieza y tablero e imprime fila por fila de acuerdo a los bits encendidos.	bitTablero, bitPieza
intentarRot	bool		Crea una pieza de prueba, llama a una función para que la rote, si no hay colisión copia los valores de esta pieza prueba a la original fila a fila.	rotar90
rotar90	void	*piezaOrg,* piezaRot	Apaga la pieza rotada, hace cambio de coordenadas de pieza en sus bytes, llama a función para ver si hay bit encendido en la pieza, hace la operatividad para la rotación y luego llama a una función para encender los bits.	bitPieza, encenderBitpieza, apagarPieza
encenderBitpieza	void	*pieza, fila, col	De acuerdo a la posición suministrada se cambia el contenido de esa coordenada de acuerdo con una operación a nivel de bits y una mascara	-
apagarPieza	void	*pieza	Se cambian todas las filas de pieza por ceros	-
bajar	bool	**tablero, alto, ancho,	Intenta mover la pieza abajo si no puede, la fija, limpia	colision, limpiarfilas, fijarpieza,generarPieza

		*pieza, &tipo, &filapieza, &colpieza, &gameOver	filas, genera nueva pieza y verifica fin de juego.	
fijarPieza	void	**tablero, alto, ancho, *pieza, fila, columna	Inserta la pieza en el tablero activando los bits correspondientes según su posición.	bitPieza, onBitTablero
onBitTablero	void	**tablero, fila, col	Activa un bit específico en el tablero usando una máscara para marcar una celda ocupada.	-
limpiarfilas	void	**tablero, alto, ancho	Elimina filas completas, desplaza las superiores hacia abajo y crea una nueva fila vacía arriba.	filallena
filallena	bool	*fila, ancho	Verifica si todos los bits de una fila están activos (fila completamente llena).	-
iniciarSemilla	void	-	Inicializa la semilla para números aleatorios.	-

6. PROBLEMAS ENCONTRADOS

Durante las diferentes etapas de análisis, implementación y desarrollo se encontraron diferentes problemas:

- La interpretación de los decimales en números binarios: en un principio se pensó que como se podía trabajar con operaciones a nivel de bits en un entero, debíamos poner su valor entero para cada pieza, sin embargo, aunque esto es verdadero se optó finalmente por usar variables tipo unsigned char, ya que permiten trabajar con lo justo y necesario que se requiere (8 bits). Para que la variable pueda leer el valor como el número binario “crudo” se usa la notación 0b al principio de cada número binario, la cual es admitida por el lenguaje C++, de esta manera se puede trabajar mejor con las operaciones binarias y resulta mucho más eficiente en memoria y en la lógica al momento de programar.

- Debido a la elección de un algoritmo numérico para la rotación a 90 grados a la derecha de la matriz pieza, pasa que cuando rota una vez, y se quiere volver a su posición inicial esta queda desplazada hacia la derecha en un factor de 2, debido a las columnas que se trasladan de acuerdo con la rotación.